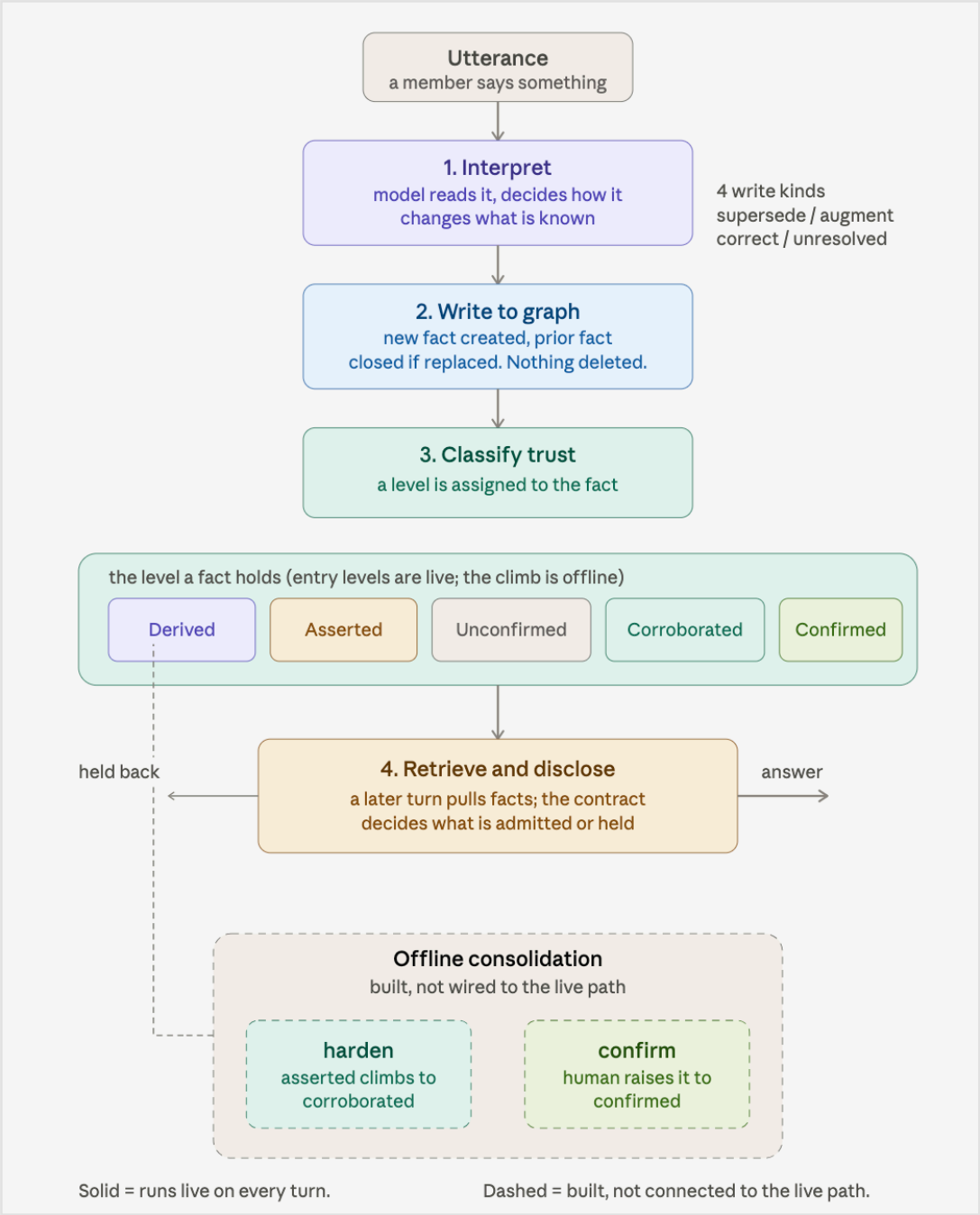
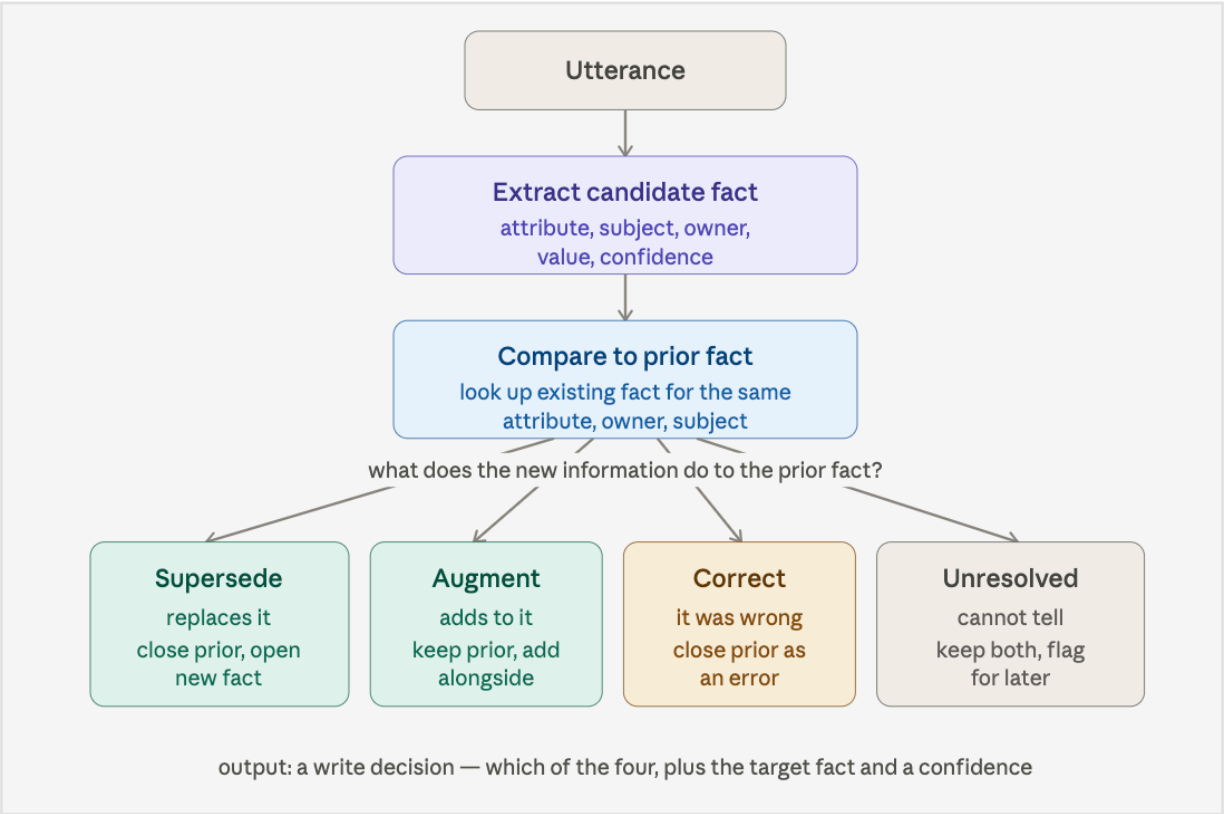


Part 1 — Interpretation, as designed. An utterance is extracted into a structured candidate (attribute, subject, owner, value, confidence), compared against any existing fact for that same key, and the *relationship* between new and old determines one of four write decisions:



Supersede — the new info replaces the old ("Ray switched to Jardiance"). Close the prior, open a new fact.
Augment — the new info adds alongside ("Ray also takes aspirin"). Keep the prior, add another.
Correct — the old was wrong ("actually, it was never metformin"). Close the prior as an error.
Unresolved — can't tell, or confidence too low. Keep both, flag for later.

Supersede — "Ray switched from metformin to Jardiance."
Prior fact: Ray takes metformin. New info replaces it. Design: close the metformin fact (valid_to set, marked superseded), open the Jardiance fact. The old fact isn't deleted — it's closed and walkable in lineage. Use case: a medication change. The system must not hold both as current, or an answer to "what does Ray take" would name a drug he stopped.
Augment — "Ray also takes a baby aspirin daily."
Prior fact: Ray takes Jardiance. New info adds, doesn't replace. Design: prior stays current, new fact opens alongside. Both are true simultaneously. Use case: a second medication. The system must keep both, or it would drop one of two current, correct facts.
Correct — "Actually, I was wrong, Ray never took metformin — that was his brother."

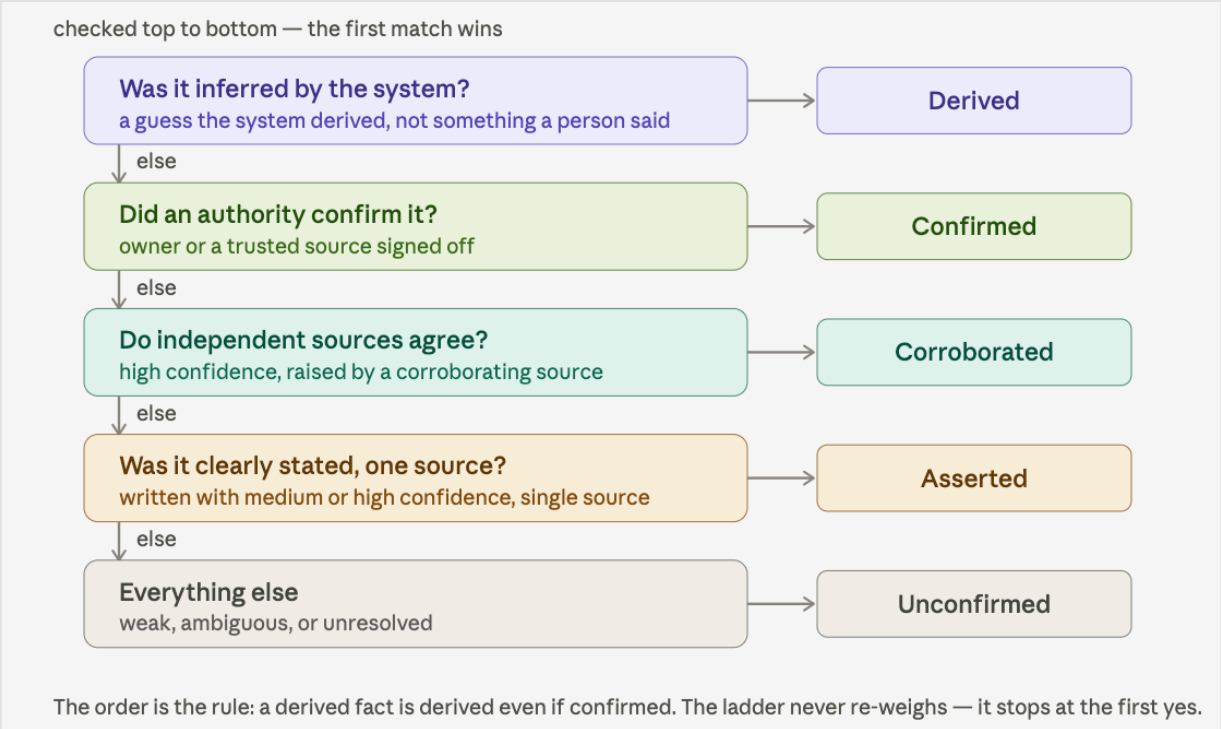
Prior fact: Ray takes metformin, recorded as if true. New info says the prior was an *error*, not a change. Design: close the prior fact as an error (distinct from superseded — record_closed_at set, closed_reason=error), open the corrected fact inheriting the original's valid_from. Use case: correcting a mistaken record. The distinction matters legally/audit-wise: "was wrong" is different from "changed," and the system records which.
Unresolved — "I think Ray's dose might have changed but I'm not sure."
Prior fact: Ray takes Jardiance 10mg. New info conflicts but confidence is too low to act. Design: don't close the prior, don't confidently write the new — keep both valid, cap confidence low, flag for later confirmation. Use case: ambiguous or hedged input. The system must not overwrite a good fact on a guess, nor silently ignore a possible change — it holds the tension and marks it.

the system distinguishes a change from a correction from an addition from an ambiguity — and records which, with an audit trail — instead of just overwriting.

Part 2 - The write is **bitemporal**: every fact carries when it was *true* (valid_from/valid_to) and when it was *recorded*. Nothing is deleted — facts get *closed*, not removed. Each write decision does something different:

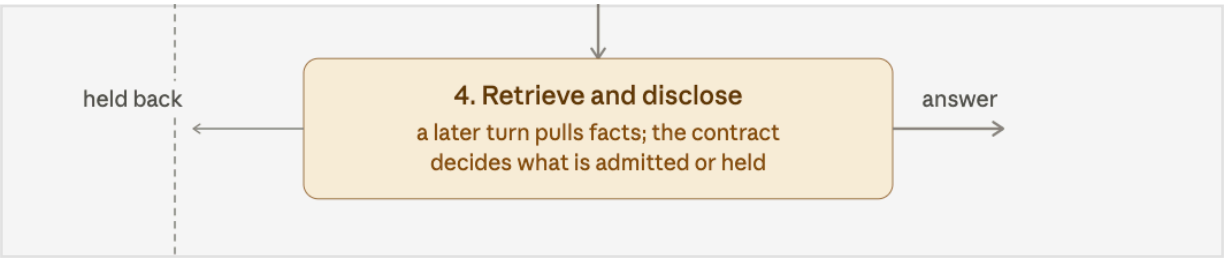
Supersede — *"Ray switched to Jardiance."* Prior metformin fact: valid_to = now, closed_reason = superseded, superseded_by → new fact. New Jardiance fact opens (valid_to = null). Two rows, one current.
Augment — *"Ray also takes aspirin."* Prior untouched, stays current. New fact opens alongside. Two rows, both current.
Correct — *"Ray never took metformin, that was wrong."* Prior: valid_to = now, closed_reason = **error** (not superseded), record_closed_at set. New fact inherits the *original's* valid_from — because it was always true, the record was just wrong.
Unresolved — *"Dose might have changed, not sure."* Both stay valid_to = null (both current), new fact confidence capped low, flagged for confirmation.
The governance point in one line: **the graph is append-only and time-aware — a change, a correction, an addition, and an ambiguity each leave a different, auditable trace.** You can always reconstruct what was believed, when, and why it changed.

Piece 3 — Classify. The key design idea: trust isn't computed by weighing factors — it's a **ladder of checks, first match wins, evaluated in a fixed order.** The fact falls through until one condition catches it.

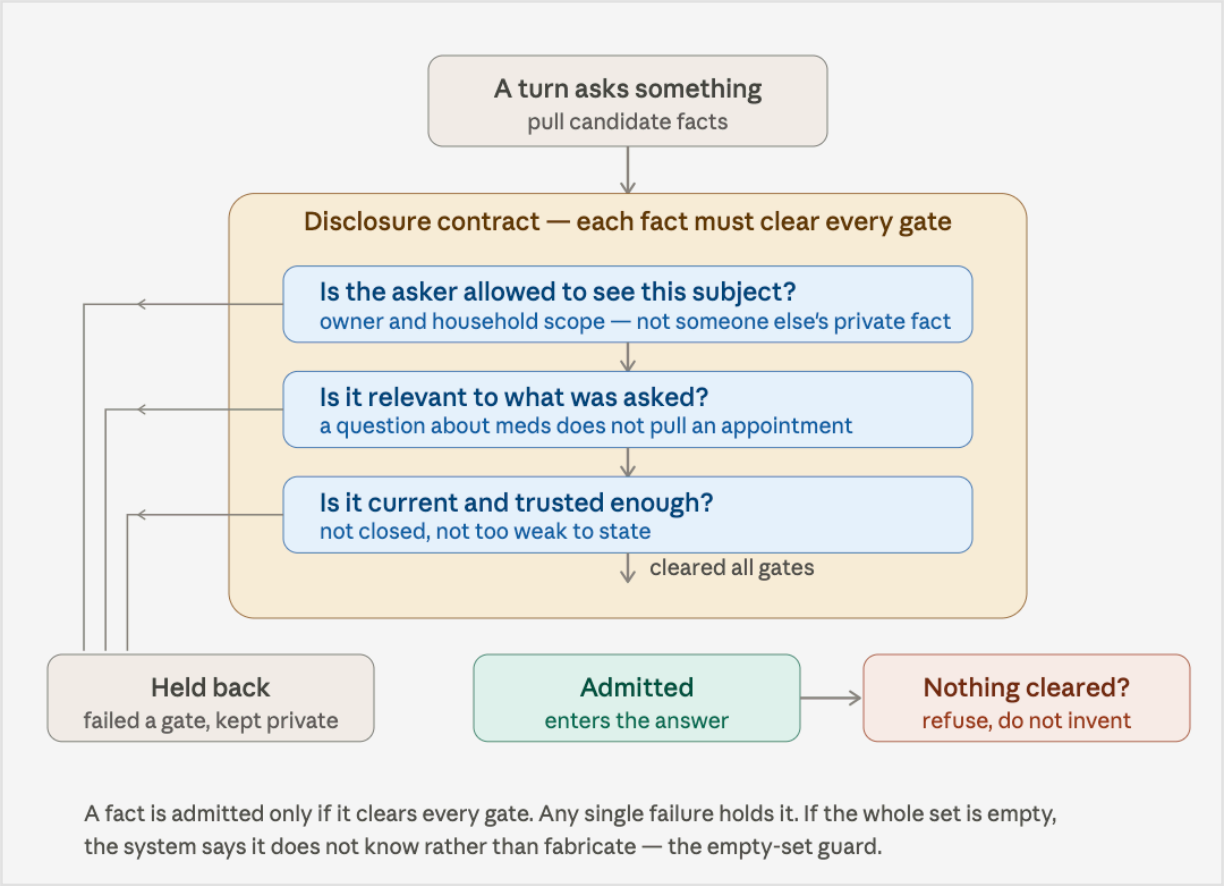


Derived — *"Dad has an elevated fall-risk pattern."* The system *inferred* this from other facts (his fall, his meds), no one stated it. Checked first, and it wins over everything — a derived fact stays derived even if later confirmed. Design intent: system inferences are permanently second-class, never laundered into higher trust.
Confirmed — *Maya's cardiology appointment, verified against the clinic.* An authority signed off. Top of the ladder. Use case: a fact you'd act on without hesitation.
Corroborated — *Ray's metformin, agreed by Maya and the clinic records.* Two independent sources, confidence raised by the agreement. Use case: strong but not authority-signed — trusted because multiple sources concur.
Asserted — *"Ray switched to Jardiance"* the moment Maya says it. One source, clearly stated, medium/high confidence. Use case: a fresh single-source claim — believed, but not yet corroborated. (This is exactly why the supersede demo shows Jardiance as *asserted*, not corroborated — it's new and single-source.)
Unconfirmed — *"I think Ray's dose maybe changed."* Weak, ambiguous, or unresolved. The catch-all. Use case: hedged input the system holds but won't rely on.
The design point in one line: **the order is the governance.** The ladder never averages or re-weighs — it stops at the first match. That's what makes trust *deterministic and explainable*: every fact can state exactly which rung caught it and why. A system that scored trust with a weighted blend couldn't tell you that.

Piece 4 — Retrieve and Disclose, with use cases. Every fact must clear **every** gate; one failure holds it.



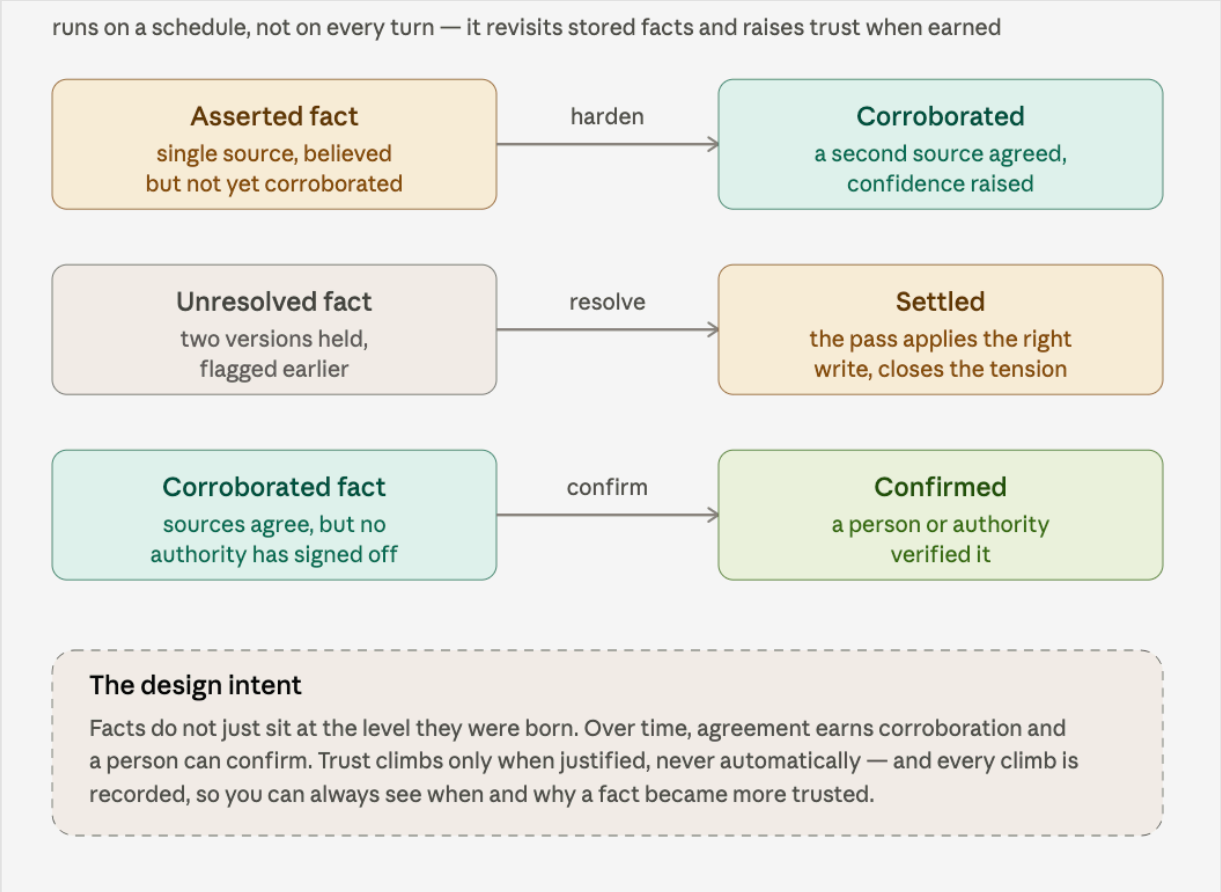
Retrieve and Disclose. This is the governance heart. A later turn asks something; the system pulls candidate facts, then runs each through the disclosure contract — a series of gates that decide admit or hold. First failed gate holds the fact.



Subject-scope — *Maya asks "what medications does Ray take?"* Ray's meds clear (Maya owns that subject). But Maya's *own* cardiology appointment is held — the question was about Ray, so her unrelated private facts don't get pulled in. Use case: cross-subject privacy. This is the gate that fires in your supersede demo.
Relevance — *Maya asks about medications.* The household trash-pickup fact is current and she's allowed to see it, but it's irrelevant to a meds question — held. Use case: don't dump everything the asker *could* see; disclose only what the question needs.

Current and trusted — *A superseded fact* (old metformin, now closed) never surfaces — it's not current. A too-weak unconfirmed fact may be held rather than stated as if reliable. Use case: don't answer with stale or flimsy facts.
Empty-set guard — *Maya asks "what's Ray's blood type?"* and nothing is on file. Nothing clears the gates because there's nothing to clear. Design: the system **refuses** — "I don't have that" — rather than inventing a plausible answer. Use case: the anti-hallucination beat. This is the strongest governance moment: an AI that says "I don't know" when it doesn't.

Piece 5 — Consolidation. This is the offline process that lets a fact *climb* trust over time. In the spec it runs periodically; in the current build it's built but not wired live. Here's how it's supposed to work.



Harden — *Maya says "Ray takes metformin" (asserted). A week later the clinic records confirm the same.* The consolidation pass notices two independent sources now agree, raises confidence, and the fact climbs asserted → corroborated. Use case: trust earned by corroboration over time, without a human doing anything.
Resolve — *Earlier, "Ray's dose maybe changed" was held as unresolved (both versions kept).* Later a clear statement arrives. The pass settles it — applies the right write (supersede the old dose), closes the tension. Use case: ambiguity that

couldn't be decided in the moment gets decided later, deterministically. **Confirm** — *Ray's metformin is corroborated. Maya (or an authority) explicitly verifies it.* The fact climbs corroborated → confirmed, the top of the ramp. Use case: the human-in-the-loop step — the only path to the highest trust.

The design intent, one line: **facts aren't frozen at birth — trust climbs when agreement or verification earns it, never automatically, and every climb is recorded.** That's what makes the system's confidence *move* over time in a defensible way.

That completes the full spec'd epistemic process, end to end: **Interpret → Write → Classify → Retrieve/Disclose → Consolidate.** Five pieces, each with use cases. You now have the reference you asked for — how the process is *supposed* to work.